tripleten

Choosing Evaluation Metric

Evaluation metrics

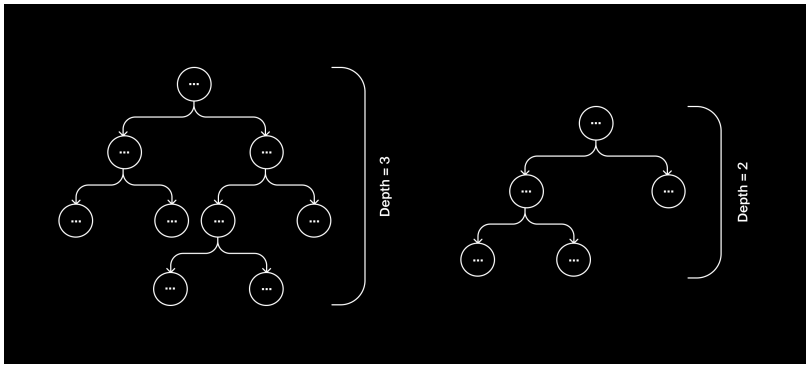
Evaluation metrics are used to measure the quality of a model and can be expressed numerically. Always make sure that your model performs **better than chance** — that is, do a **sanity check**.

Underfitting and overfitting

Underfitting occurs when the *accuracy* is low and approximately the same for the training set and the test set.

It is not always possible to avoid overfitting or underfitting. When you improve one, the risk having the other increases.

The depth of the tree (**height)** is the maximum number of conditions from the "top" of the tree to the final answer based on the number of node-to-node transitions.



The depth of the tree in *sklearn* can be set by the `max_depth` parameter:

COPY PYTHON

```
# specify the depth (unlimited by default)
```

```
model = DecisionTreeClassifier(random_state=54321, max_depth=3)

model.fit(features, target)
```

Classification Task Metrics

Accuracy

The ratio of the number of correct answers to the total number of questions (i.e., the test dataset size) is called **accuracy.**

To calculate *accuracy*, use this formula:

$$\text{accuracy} = \frac{\overbrace{\text{total number of questions} - \text{number of errors}}^{\text{number of correct answers}}}{\text{total number of questions}}$$

COPY PYTHON

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(target, predictions)
```

Precision/Recall

- **Precision** takes all apartments that were deemed expensive by the model (they're marked as "1") and calculates what fraction of them was actually expensive. The apartments that weren't recognized by the model are ignored.
- **Recall** takes all apartments that are actually expensive and calculates what fraction of them was recognized by the model. Apartments that were recognized by the model by mistake are ignored.

Regression Task Metrics

Mean Square Error (MSE)

The most commonly used evaluation metric for regression tasks is the **Mean Squared Error or MSE.**

To find MSE, calculate the error of each observation first:

$$\text{Observation error} = \text{Model prediction} - \text{Correct answer}$$

Calculate MSE using the following formula:

$$\text{MSE} = \frac{\text{Sum of the squares of the observations' errors}}{\text{Number of observations}}$$

Let's analyze these calculations:

1. The observation error shows the extent of the discrepancy between the correct answer and the prediction. If the error is much greater than zero, the model has overpriced the apartment; if the error is much less than zero, then the model underpriced it.
2. It would be pointless to add up the errors as they are, since positive errors would cancel out negative ones. To make them all count, we need to get rid of the signs by squaring each of them.
3. We find the mean to obtain data for all the observations.

MSE should be as low as possible.

Calculating the mean of squared errors (MSE) and the square root of the mean of the squared errors (RMSE) with sklearn.

COPY PYTHON

```python
from sklearn.metrics import mean_squared_error

mse = mean_squared_error(answers, predictions)
rmse = mse ** 0.5
```

MSE calculation

To calculate mean squared error, import the `mean_squared_error()` function from `sklearn.metrics` module.

COPY PYTHON

```python
from sklearn.metrics import mean_squared_error
```

```
2
3 mse = mean_squared_error(answers, predictions)
```

You'll get square units (for example, "square dollars"). To get an evaluation metric in the regular units, find the square root of MSE. Then, you will get RMSE (root mean squared error):

COPY PYTHON

```
1 rmse = mse ** 0.5
```

Evaluation metrics in Scikit-Learn

The metric functions of the scikit-learn library can be found in the **metrics** module. Use the `accuracy_score()` function to calculate the *accuracy*.

COPY PYTHON

```
1 from sklearn.metrics import accuracy_score
```

The function takes two arguments (correct answers and model predictions) and returns the *accuracy* value.

COPY PYTHON

```
1 accuracy = accuracy_score(target, predictions)
```